

Docker-03

- * Create a image from running container.
- * Copy image from local machine to docker server and load the image.
- * Create Docker image using alpine and customize with tomcat.
 1. Create single stage and multi stage docker file using this source code: <https://github.com/betawins/multi-stage-example.git>
 2. Install docker compose and execute sample application.
 3. Implement solution to scan images when pushed to docker registry.
 4. Implement solution to scan images when pushed to AWS ECR.
 5. Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

1* Create a image from running container.

Step 1: List running containers

`docker ps`

This will show all currently running containers.

tep 2: Commit the running container as an image

Use this command:

`docker commit <container_id_or_name> <new_image_name>:<tag>`

Step 3: Verify the new image

`docker images`

```
[root@ip-172-31-20-146 ~]# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
9afdc5c06e2b   nginx    "/docker-entrypoint..." 4 seconds ago  Up 3 seconds  0.0.0.0:81->80/tcp, :::81->80/tcp  strange_fermat
sha256:83b9f4c22af3bbbedb62a29d398f2cd663d48842f7cd8fd0cddc02ad822ebb885
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx-custom  v1        83b9f4c22af3   4 seconds ago  161MB
nginx         latest    5cdef4ac3335   34 hours ago   161MB
[root@ip-172-31-20-146 ~]#
```

2. Copy image from local machine to docker server and load the image.

Step 1: Pull image on local machine

```
docker pull nginx
```

✓ Downloads `nginx:latest` on local machine.

```
connection to ssl is 01 15 10174 not on 17compute.amazonaws.com closed.  
waseemakram@waseem Downloads % docker pull nginx  
Using default tag: latest  
latest: Pulling from library/nginx  
58072fb4fd7c: Pull complete  
5380ffdd215e: Pull complete  
1063666c8b8e: Pull complete  
fb8853dab2da: Pull complete  
0e9ba714bdf1: Pull complete  
3ea009573b47: Pull complete  
efc7865cf13e: Pull complete  
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208  
Status: Downloaded newer image for nginx:latest  
docker.io/library/nginx:latest
```

Step 2: Verify image exists locally

```
docker images
```

✓ Confirms `nginx:latest` is available.

```
waseemakram@waseem Downloads % docker images  
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE  
nginx                latest      341bf0f3ce6c  34 hours ago  255MB
```

Step 3: Save Docker image as a tar file

```
docker save nginx:latest -o nginx.tar
```

✓ Converts Docker image into a portable file.

```
waseemakram@waseem Downloads % docker save nginx:latest -o nginx.tar  
waseemakram@waseem Downloads % ls -lh nginx.tar  
-rw-----@ 1 waseemakram  staff    58M  6 Feb 15:22 nginx.tar
```

Verify tar file creation

```
ls -lh nginx.tar
```

✓ Confirms image archive exists (≈58MB).

Step 4: Copy image to Docker server (using SSH key)

```
scp -i ansible.pem nginx.tar ec2-user@13.61.143.101:/home/  
ec2-user/
```

✓ Securely copies image from local machine to Docker server.

```
Step: connection closed  
waseemakram@waseem Downloads % scp -i ansible.pem nginx.tar ec2-user@13.61.143.101:/home/ec2-user/  
nginx.tar  
waseemakram@waseem Downloads %
```

Step 5: Login to Docker server

```
dockerfile index.html
[root@ip-172-31-20-146 ~]# exit
logout
[ec2-user@ip-172-31-20-146 ~]$ ls
nginx.tar
[ec2-user@ip-172-31-20-146 ~]$ ll
total 59848
-rw-----. 1 ec2-user ec2-user 61280768 Feb  6 09:56 nginx.tar
[ec2-user@ip-172-31-20-146 ~]$
```

Load image on Docker server

`Sudo docker load -i nginx.tar`

✓ Imports the image into Docker.

Step 6: Verify image on Docker server

`Sudo docker images`

✓ Confirms `nginx:latest` is available on server.

```
[ec2-user@ip-172-31-20-146 ~]$ sudo docker load -i nginx.tar
a0e71ab2b234: Loading layer [====>] 30.14MB/30.14MB
252a1aff40cb: Loading layer [====>] 31.1MB/31.1MB
92956b4f6a0e: Loading layer [====>] 626B/626B
c8b198af1b7f: Loading layer [====>] 955B/955B
d98fe3382c25: Loading layer [====>] 404B/404B
84f0094d885e: Loading layer [====>] 1.208kB/1.208kB
b3ad8aedf056: Loading layer [====>] 1.396kB/1.396kB
The image nginx:latest already exists, renaming the old one with ID sha256:5cdef4ac3335f68428701c14c5f12992f5e3669ce8a
Loaded image: nginx:latest
[ec2-user@ip-172-31-20-146 ~]$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
nginx-custom         v1                 83b9f4c22af3       19 minutes ago     161MB
nginx                <none>             5cdef4ac3335       34 hours ago       161MB
nginx                latest            2af158aaca82       34 hours ago       181MB
[ec2-user@ip-172-31-20-146 ~]$
```

i-0cbbcd486344f6310 (docker-ec2)

nginx-custom	v1	83b9f4c22af3	161MB	
nginx	<none>	5cdef4ac3335	161MB	← old dangling image
nginx	latest	2af158aaca82	181MB	← newly loaded image

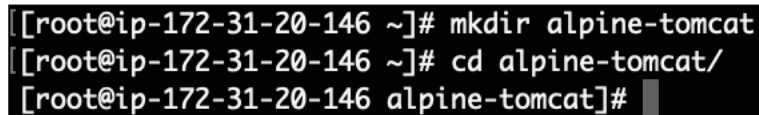
3. Create Docker image using alpine and customize with tomcat.

We will:

1. Use **Alpine Linux** (lightweight)
2. Install **Java**
3. Install **Apache Tomcat**
4. Create a **custom Docker image**
5. Verify by running it

STEP 1: Create a working directory

```
mkdir alpine-tomcat  
cd alpine-tomcat
```

A terminal window with a black background and white text. It shows three lines of commands and their outputs: 1. '[root@ip-172-31-20-146 ~]# mkdir alpine-tomcat' followed by a new line. 2. '[root@ip-172-31-20-146 ~]# cd alpine-tomcat/' followed by a new line. 3. '[root@ip-172-31-20-146 alpine-tomcat]# ' followed by a cursor bar.

```
[root@ip-172-31-20-146 ~]# mkdir alpine-tomcat  
[root@ip-172-31-20-146 ~]# cd alpine-tomcat/  
[root@ip-172-31-20-146 alpine-tomcat]# █
```

STEP 2: Create Dockerfile

```
vi Dockerfile
```

—-> write the below code data

```

FROM alpine:3.19

# Install Java and required tools
RUN apk add --no-cache openjdk11 wget tar bash

# Set working directory
WORKDIR /opt

# Download and extract Tomcat (YOUR URL)
RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.115/bin/apache-tomcat-9.0.115.tar.gz \
    && tar -xzf apache-tomcat-9.0.115.tar.gz \
    && mv apache-tomcat-9.0.115 tomcat \
    && rm apache-tomcat-9.0.115.tar.gz

# Expose Tomcat port
EXPOSE 8080

# Start Tomcat
CMD ["/opt/tomcat/bin/catalina.sh", "run"]

```

—> save and exit

Build the image

```
docker build -t alpine-tomcat:latest .
```

```

[root@ip-172-31-20-146 alpine-tomcat]# vi Dockerfile
[root@ip-172-31-20-146 alpine-tomcat]# docker build -t alpine-tomcat:latest .
[+] Building 1.5s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 593B
=> [internal] load metadata for docker.io/library/alpine:3.19
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/alpine:3.19@sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a3145e
=> CACHED [2/4] RUN apk add --no-cache openjdk11 wget tar bash
=> CACHED [3/4] WORKDIR /opt
=> [4/4] RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.115/bin/apache-tomcat-9.0.115.tar.gz    && t
=> exporting to image
=> => exporting layers
=> => writing image sha256:1939b2f8bfda82d4b6537d3b80ccdeb25b86e6a583cb6fd8a7ffe5099e656b6e
=> => naming to docker.io/library/alpine-tomcat:latest
[root@ip-172-31-20-146 alpine-tomcat]# vi Dockerfile
[root@ip-172-31-20-146 alpine-tomcat]# docker images

```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
alpine-tomcat	latest	1939b2f8bfda	About a minute ago	296MB

Run the container

```
docker run -d -p 8080:8080 --name tomcat alpine-tomcat:latest
```

```
[root@ip-172-31-20-146 alpine-tomcat]# docker ps
```

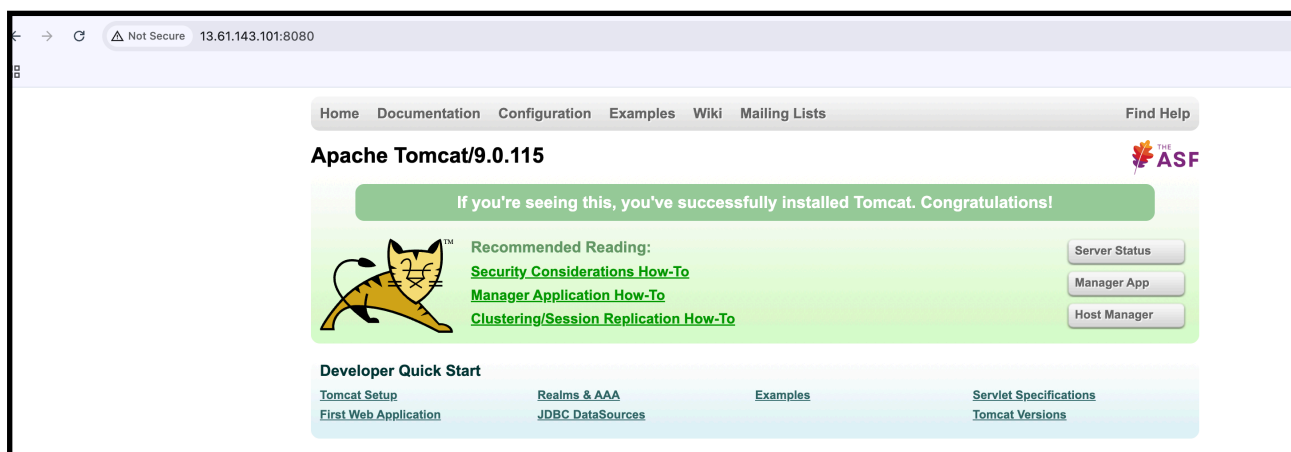
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
11d6355e8959	alpine-tomcat:latest	"/opt/tomcat/bin/cat..."	13 seconds ago	Up 12 seconds	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp	tomcat

Verify :

open in browser:

http://<SERVER_IP>:8080

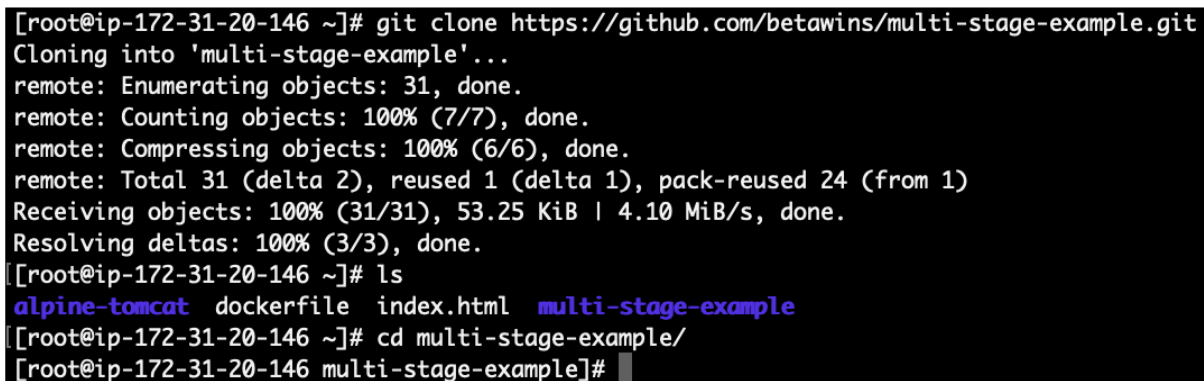
You should see the **Apache Tomcat welcome page**



4. Create single stage and multi stage docker file using this source code: <https://github.com/betawins/multi-stage-example.git>

STEP 1: Clone the source code

```
git clone https://github.com/betawins/multi-stage-example.git
```



```
[root@ip-172-31-20-146 ~]# git clone https://github.com/betawins/multi-stage-example.git
Cloning into 'multi-stage-example'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 31 (delta 2), reused 1 (delta 1), pack-reused 24 (from 1)
Receiving objects: 100% (31/31), 53.25 KiB | 4.10 MiB/s, done.
Resolving deltas: 100% (3/3), done.
[root@ip-172-31-20-146 ~]# ls
alpine-tomcat  dockerfile  index.html  multi-stage-example
[root@ip-172-31-20-146 ~]# cd multi-stage-example/
[root@ip-172-31-20-146 multi-stage-example]#
```

```
cd multi-stage-example
```

—> Now you are inside the application source code folder.

PART A: Single-Stage Dockerfile

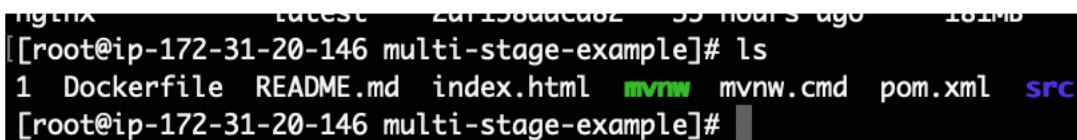
What is single-stage?

- Build the application
- Run the application
 - > **Everything happens in ONE image**

SINGLE-STAGE DOCKER — STEPS

—> Step 1: Go to project directory

```
cd multi-stage-example
```

A terminal window screenshot showing the command 'ls' being executed in a directory named 'multi-stage-example'. The output lists files: '1 Dockerfile', 'README.md', 'index.html', 'mvnw' (highlighted in green), 'mvnw.cmd', 'pom.xml', and 'src' (highlighted in blue). The prompt is '[root@ip-172-31-20-146 multi-stage-example]#'.

```
nginx latest 2d158ddca02 55 hours ago 181MB  
[root@ip-172-31-20-146 multi-stage-example]# ls  
1 Dockerfile README.md index.html mvnw mvnw.cmd pom.xml src  
[root@ip-172-31-20-146 multi-stage-example]#
```

Step 2: Create / edit Dockerfile (single-stage)

vi Dockerfile

Paste this **single-stage Dockerfile**:

```
FROM maven:3.9.6-eclipse-temurin-8
```

```
WORKDIR /app
```

```
COPY . .
```

```
RUN mvn clean package -DskipTests
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
```

Save and exit.

```
FROM maven:3.9.6-eclipse-temurin-8

WORKDIR /app

COPY . .

RUN mvn clean package -DskipTests

EXPOSE 8080

CMD ["java", "-jar", "target/multi-stage-example-0.0.1-SNAPSHOT.jar"]
```

Step 3: Build single-stage Docker image

```
docker build -t single-stage:v1 .
```

```
[root@ip-172-31-20-146 multi-stage-example]# docker build -t single-stage:v1 .
[+] Building 26.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 276B
=> [internal] load metadata for docker.io/library/maven:3.9.6-eclipse-temurin-8
=> [auth] library/maven:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/maven:3.9.6-eclipse-temurin-8@sha256:c85a28e9be6552408174e726d617a451350576f157aa5b542252efc128055b96
=> => resolve docker.io/library/maven:3.9.6-eclipse-temurin-8@sha256:c85a28e9be6552408174e726d617a451350576f157aa5b542252efc128055b96
=> => sha256:468c4e4f3f3bf41ce8a32ef250abad42aa22ba62f8c81be1f5bc0fac7f55b52 2.41kB / 2.41kB
=> => sha256:7c089f6b0c6a61401ec8a2814be7abdf619f5acae5e2036c867b746c89c3e16b 7.45kB / 7.45kB
=> => sha256:dce394e5c05f6275f1a3d93ef078caadf4c6e88066e708ffa5cea964ded0c3c2 12.91MB / 12.91MB
=> => sha256:b9e985ee26d7966950debab0bf9d7e17e8d0a9cc5b9b36f2d3066975605faa23 103.60MB / 103.60MB
=> => sha256:c85a28e9be6552408174e726d617a451350576f157aa5b542252efc128055b96 992B / 992B
=> => sha256:4a023cab5400feb5c1ab725beb8345ddb0e3200314004b56677a5eee2e8c86cf 30.44MB / 30.44MB
=> => sha256:b63f033143623e93ed39f65fc99423eb715bdb5bdfec213fd8173f9b6e813a43 159B / 159B
=> => extracting sha256:4a023cab5400feb5c1ab725beb8345ddb0e3200314004b56677a5eee2e8c86cf
=> => sha256:95775e4b4d5153b3651e53c635338b8c37d6cd12fb3dff3f6df1534464e41628 734B / 734B
=> => sha256:a3e6e652780e6753d98b0da3044598be68fde203a1b07b67d768828c87daa28b 18.99MB / 18.99MB
=> => sha256:1adc1fae955ec831a61f0a1263fd2f6628e34724c79a1a738eafc951a79d29b6 9.48MB / 9.48MB
=> => sha256:04622b7dd6f2b793d4d7d201fc47d07b47fc878d7de1c5bf0ecd18504655c8f7 853B / 853B
=> => extracting sha256:dce394e5c05f6275f1a3d93ef078caadf4c6e88066e708ffa5cea964ded0c3c2
=> => sha256:27118d384765f7d78f03a77797a6efc4b34d36c9fa3b4a2c0378a1dc4c3e7cd5 358B / 358B
=> => sha256:2a45d918a7b64dded6b9ff3fe97cc9efb5e65700f45494556065564d204bcde8 156B / 156B
=> => extracting sha256:b9e985ee26d7966950debab0bf9d7e17e8d0a9cc5b9b36f2d3066975605faa23
=> => extracting sha256:b63f033143623e93ed39f65fc99423eb715bdb5bdfec213fd8173f9b6e813a43
=> => extracting sha256:95775e4b4d5153b3651e53c635338b8c37d6cd12fb3dff3f6df1534464e41628
=> => extracting sha256:a3e6e652780e6753d98b0da3044598be68fde203a1b07b67d768828c87daa28b
=> => extracting sha256:1adc1fae955ec831a61f0a1263fd2f6628e34724c79a1a738eafc951a79d29b6
=> => extracting sha256:04622b7dd6f2b793d4d7d201fc47d07b47fc878d7de1c5bf0ecd18504655c8f7
=> => extracting sha256:27118d384765f7d78f03a77797a6efc4b34d36c9fa3b4a2c0378a1dc4c3e7cd5
=> => extracting sha256:2a45d918a7b64dded6b9ff3fe97cc9efb5e65700f45494556065564d204bcde8
=> [internal] load build context
=> => transferring context: 7.67kB
=> [2/4] WORKDIR /app
=> [3/4] COPY . .
=> [4/4] RUN mvn clean package -DskipTests
=> exporting to image
=> => exporting layers
=> => writing image sha256:84265953cf40f8ca9ace4616224a82573ec280d58ac9d0327816df6a6cae85fc
=> => naming to docker.io/library/single-stage:v1
[root@ip-172-31-20-146 multi-stage-example]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
single-stage	v1	84265953cf40	14 seconds ago	465MB

Step 4: Verify Docker image

`docker images`

Note the **image size (MB)**.

```
=> => naming to docker.io/library/single-stage:v1
[root@ip-172-31-20-146 multi-stage-example]# docker images
REPOSITORY      TAG        IMAGE ID      CREATED        SIZE
single-stage     v1         84265953cf40  14 seconds ago 465MB
```

465MB

Step 5: Run single-stage container

`docker run -d -p 8081:8080 single-stage:v1`

```
[root@ip-172-31-20-146 multi-stage-example]# docker run -d -p 8081:8080 single-stage:v1
675e46f474e23aed91d9fb0e6bf07e1a40dd3ebcb0d5d156ae24dbc402678c57
[root@ip-172-31-20-146 multi-stage-example]# vi Dockerfile
```

Step 6: Verify container

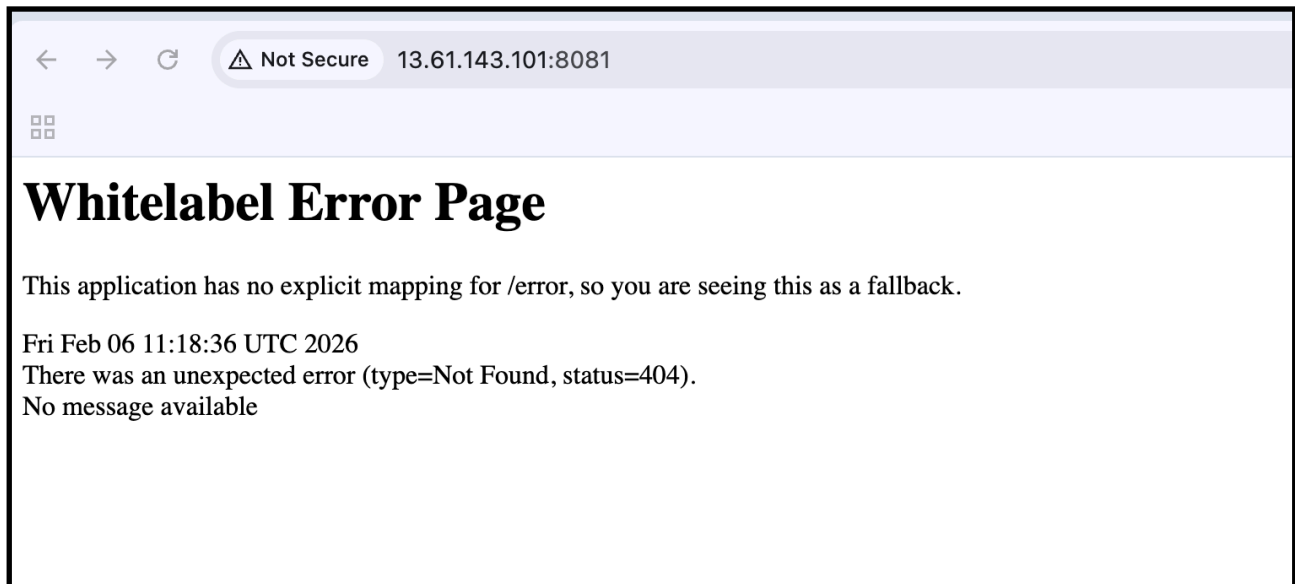
`docker ps`

```
[root@ip-172-31-20-146 multi-stage-example]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
675e46f474e2   single-stage:v1 "/usr/local/bin/mvn-..." 9 minutes ago  Up 9 minutes  0.0.0.0:8081->8080/tcp, :::8081->8080/tcp  affectionate_archimedes
```

Step 7: Verify application

Open browser:

http://<SERVER_IP>:8081



(Whitelabel page = app running)

Step 8: Record observation

docker images

Write:

- **Single-stage image size: 465MB**
-

Part:B

MULTI-STAGE DOCKER — STEPS

Step 1: Edit Dockerfile (multi-stage)

vi Dockerfile

Paste this **multi-stage Dockerfile**:

```
# ----- BUILD STAGE -----
FROM maven:3.9.6-eclipse-temurin-8 AS builder

WORKDIR /app
COPY . .

RUN mvn clean package -DskipTests

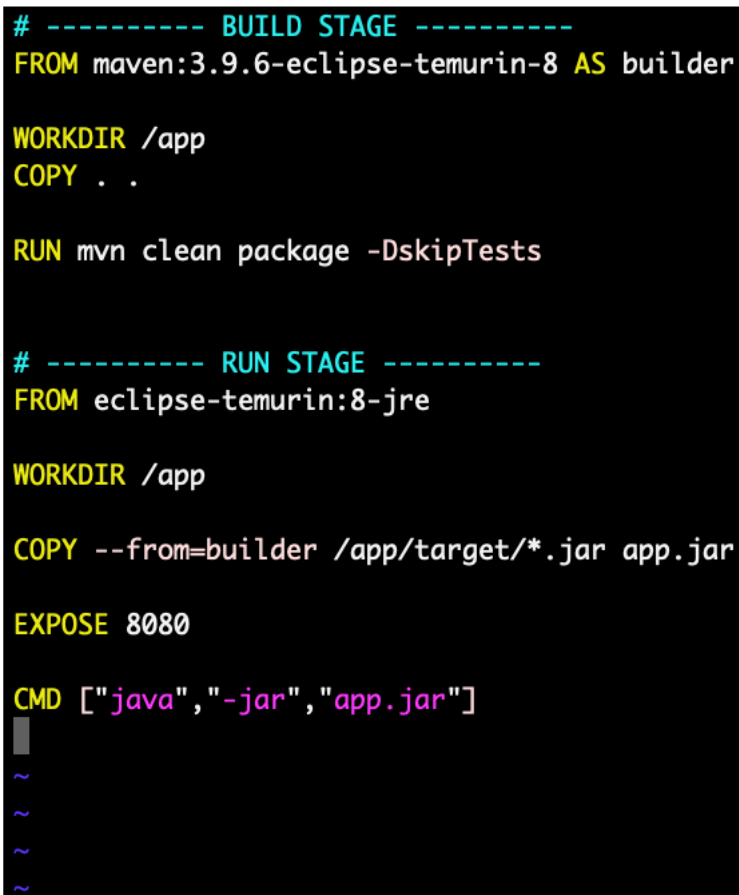
# ----- RUN STAGE -----
FROM eclipse-temurin:8-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
Save and exit.
```

A screenshot of a terminal window with a black background and white text. The text is a Dockerfile, which defines the build and run stages of a container. The build stage uses the maven:3.9.6-eclipse-temurin-8 image as a base, sets the working directory to /app, copies the source code, and runs mvn clean package -DskipTests. The run stage uses the eclipse-temurin:8-jre image as a base, sets the working directory to /app, copies the built jar file from the builder stage, and exposes port 8080. The command to run the container is java -jar app.jar. The terminal shows the Dockerfile content line by line, with some lines highlighted in yellow and green. At the bottom, there are four tilde (~) characters, indicating the end of the file.

```
# ----- BUILD STAGE -----
FROM maven:3.9.6-eclipse-temurin-8 AS builder

WORKDIR /app
COPY . .

RUN mvn clean package -DskipTests

# ----- RUN STAGE -----
FROM eclipse-temurin:8-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
~
~
~
~
```

Step 2: Build multi-stage Docker image

```
docker build -t multi-stage:v1 .
```

```
root@ip-172-31-20-146 multi-stage-example]# docker build -t multi-stage:v1 .
[+] Building 20.9s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 408B
=> [internal] load metadata for docker.io/library/eclipse-temurin:8-jre
=> [internal] load metadata for docker.io/library/maven:3.9.6-eclipse-temurin-8
=> [auth] library/eclipse-temurin:pull token for registry-1.docker.io
=> [auth] library/maven:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [builder 1/4] FROM docker.io/library/maven:3.9.6-eclipse-temurin-8@sha256:c85a28e9be6552408174e726d617a451350576f157aa5b542252efc128055b96
=> [stage-1 1/3] FROM docker.io/library/eclipse-temurin:8-jre@sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b
=> => resolve docker.io/library/eclipse-temurin:8-jre@sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b
=> => sha256:a3629ac5b9f4680dc2032439ff2354e73b06a6cc2e68f0035a2d7c001c8b4114 29.73MB / 29.73MB
=> => sha256:199d4528d3c4775b4bdc5bf4fde1b45609ba75be6d41cec44a22f4756c5f90c8 25.47MB / 25.47MB
=> => sha256:d4d7e74a8fd836219f2b267183c499a4fe82d8206aee2e83d64f88f719ddfa 42.33MB / 42.33MB
=> => sha256:b192b6bf684fd965fd1d85a44d39e6abe6f7e696b897dd2701e266ce9c3af72b 5.91kB / 5.91kB
=> => sha256:815c215bb5f18d112c486a7956ba9c327607865b179d462a3510fd32853fe260 1.94kB / 1.94kB
=> => sha256:0d427285ff2d1b127eb6929a62b94a6c96ca9f37d2974ab8eac2c4bc09c5c0b9 5.65kB / 5.65kB
=> => extracting sha256:a3629ac5b9f4680dc2032439ff2354e73b06a6cc2e68f0035a2d7c001c8b4114
=> => sha256:1cd6e14f0cb33d9c375fe3b8d66d8c18003d652497809519a509152e194c9d11 128B / 128B
=> => sha256:17c4fafac5828cb2d0471572584c7b62a70d62e78b4709fa1107f66d88cbb1f9 2.28kB / 2.28kB
=> => extracting sha256:199d4528d3c4775b4bdc5bf4fde1b45609ba75be6d41cec44a22f4756c5f90c8
=> => extracting sha256:d4d7e74a8fd836219f2b267183c499a4fe82d8206aee2e83d64f88f719ddfa
=> => extracting sha256:1cd6e14f0cb33d9c375fe3b8d66d8c18003d652497809519a509152e194c9d11
=> => extracting sha256:17c4fafac5828cb2d0471572584c7b62a70d62e78b4709fa1107f66d88cbb1f9
=> [internal] load build context
=> => transferring context: 7.80kB
=> CACHED [builder 2/4] WORKDIR /app
=> [builder 3/4] COPY . .
=> [builder 4/4] RUN mvn clean package -DskipTests
=> [stage-1 2/3] WORKDIR /app
=> [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
=> exporting to image
=> => exporting layers
=> => writing image sha256:424b6f2b8930a7945ed102ab197560008c86e7dc97a29799499ed972f167022e
=> => naming to docker.io/library/multi-stage:v1
root@ip-172-31-20-146 multi-stage-example]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
multi-stage	v1	424b6f2b8930	10 seconds ago	273MB

Step 3: Verify Docker image

docker images

Note the **image size (MB)**. = 272MB

```
=> => naming to docker.io/library/multi-stage:v1
[ root@ip-172-31-20-146 multi-stage-example]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
multi-stage	v1	424b6f2b8930	10 seconds ago	273MB
single-stage	v1	84265953cf40	5 minutes ago	465MB

Step 4: Run multi-stage container

```
docker run -d -p 8082:8080 multi-stage:v1
```

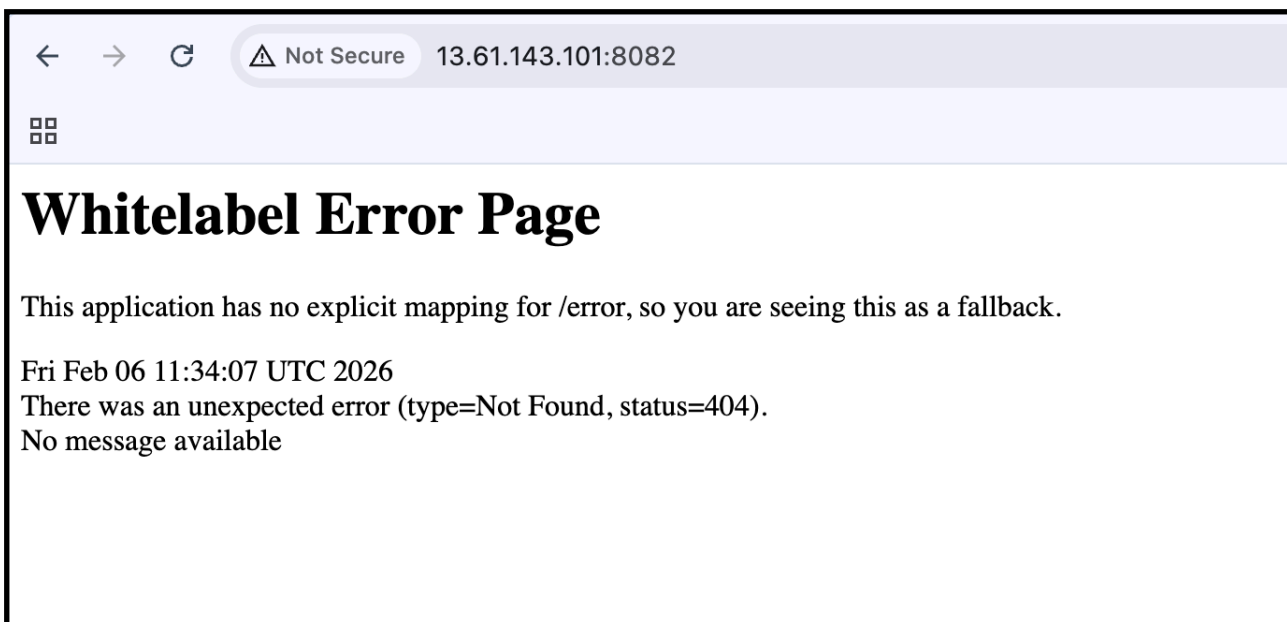
```
[root@ip-172-31-20-146 multi-stage-example]# docker run -d -p 8082:8080 multi-stage:v1
7aeac6dd2b337e33b2a2c714d2ad00ad903459e017cf4b1928d34d1f2bbb3891
[root@ip-172-31-20-146 multi-stage-example]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
7aeac6dd2b33	multi-stage:v1	"__cacert_entrypoin..."	5 seconds ago	Up 4 seconds	0.0.0.0:8082->8080/tcp, :::8082->8080/tcp	reverent_swanson
675e46f474e2	single-stage:v1	"/usr/local/bin/mvn-..."	15 minutes ago	Up 15 minutes	0.0.0.0:8081->8080/tcp, :::8081->8080/tcp	affectionate_archimedes

Step 6: Verify application

Open browser:

`http://<SERVER_IP>:8082`
(Whitelabel page = success)



tep 7: Record observation

docker images

Write:

- **Multi-stage image size: 273MB**

1. Comparison

- Compared image sizes using `docker images`.

```
=> => naming to docker.io/library/multi-stage.v1
[[root@ip-172-31-20-146 multi-stage-example]# docker images
REPOSITORY      TAG          IMAGE ID      CREATED        SIZE
multi-stage     v1          424b6f2b8930  10 seconds ago 273MB
single-stage    v1          84265953cf40  5 minutes ago  465MB
```

- Verified that **multi-stage images are smaller and more efficient** than single-stage images.

5.Install docker compose and execute sample application.

STEP 1: Check Docker is installed

`docker --version`

If Docker version is shown → OK.

```
[[root@ip-172-31-20-146 ~]# docker --version
Docker version 25.0.14, build 0bab007
[[root@ip-172-31-20-146 ~]# █
```

1) Install Docker over it.

2) Now to install docker-compose, run

```
sudo curl -L https://github.com/docker/compose/releases/download/1.21.0/docker-compose-`uname -s`-`uname -m` |  
sudo tee /usr/local/bin/docker-compose > /dev/null
```

3) For permission

```
sudo chmod +x /usr/local/bin/docker-compose
```

4) Create a symbolic link

```
ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
```

```
[root@ip-172-31-20-146 ~]# docker-compose --version  
-bash: docker-compose: command not found  
[root@ip-172-31-20-146 ~]# sudo curl -L https://github.com/docker/compose/releases/download/1.21.0/docker-compose-  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
           Dload  Upload   Total   Spent    Left   Speed  
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0  
100 10.3M 100 10.3M    0     0 15.7M      0  --:--:-- --:--:-- --:--:-- 15.7M  
[root@ip-172-31-20-146 ~]# sudo chmod +x /usr/local/bin/docker-compose  
[root@ip-172-31-20-146 ~]# ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose  
[root@ip-172-31-20-146 ~]# docker-compose --version  
docker-compose version 1.21.0, build 5920eb0  
[root@ip-172-31-20-146 ~]#
```

Steps:

Create docker-compose.yml

```
vi docker-compose.yml
```

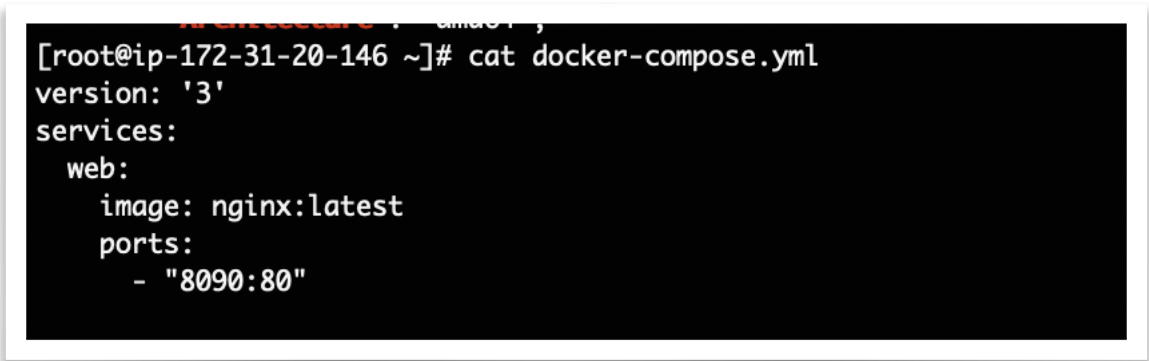
```
version: '3'  
services:
```

```
web:
  image: nginx:latest
  ports:
    - "8090:80"
```



```
version: '3'
services:
  web:
    image: nginx:latest
    ports:
      - "8090:80"
```

Save exit



```
[root@ip-172-31-20-146 ~]# cat docker-compose.yml
version: '3'
services:
  web:
    image: nginx:latest
    ports:
      - "8090:80"
```

—> Start application using Docker Compose

```
docker-compose up -d
```

—> Verify running service

```
docker-compose ps
```

```
docker ps
```

```
[root@ip-172-31-20-146 ~]# docker-compose up -d
Creating network "root_default" with the default driver
Creating root_web_1 ... done
[root@ip-172-31-20-146 ~]# docker-compose ps
```

Name	Command	State	Ports
root_web_1	/docker-entrypoint.sh nginx ...	Up	0.0.0.0:8090->80/tcp, :::8090->80/tcp

```
[root@ip-172-31-20-146 ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
31eb3cd251c8	nginx:latest	"/docker-entrypoint..."	19 seconds ago	Up 18 seconds	0.0.0.0:8090->80/tcp, :::8090->80/tcp	root_web_1

```
[root@ip-172-31-20-146 ~]# vi docker-compose.yml
[root@ip-172-31-20-146 ~]#
```

Test application

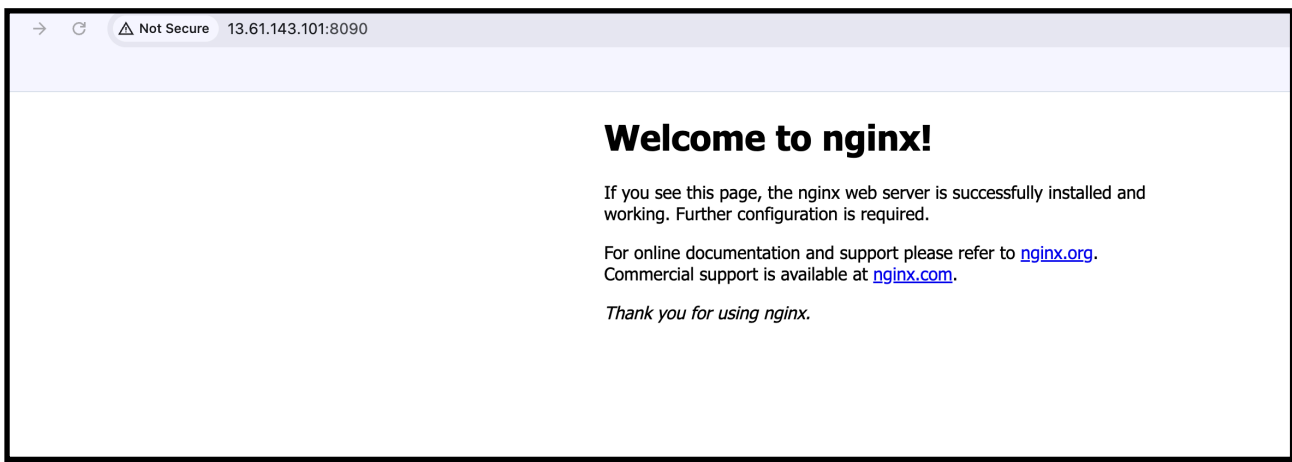
```
[root@ip-172-31-20-146 ~]# vi docker-compose.yml
[root@ip-172-31-20-146 ~]# curl http://localhost:8090
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

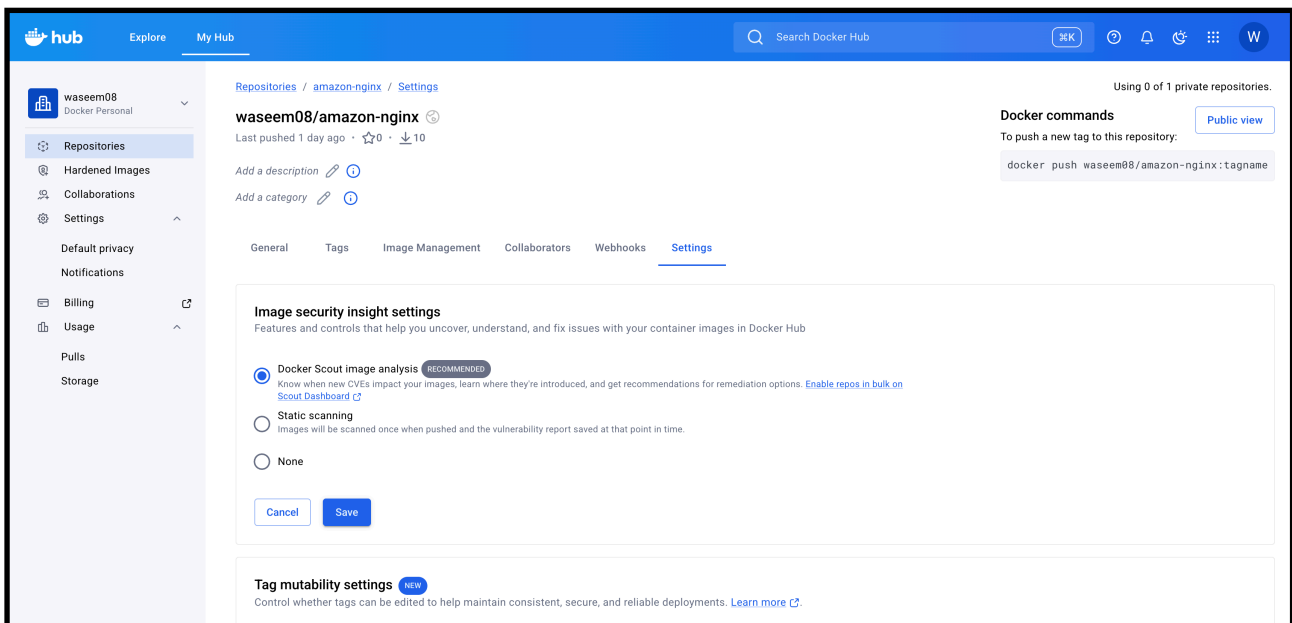
<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@ip-172-31-20-146 ~]#
```

```
curl http://localhost:8090
```



6.Implement solution to scan images when pushed to docker registry.



Step 1: Docker login

```
[root@ip-172-31-20-146 ~]# docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 ~]#
```

Step:2

—->create a tag of image

```
docker tag multi-stage:v1 waseem08/amazon-nginx:v1
```

```
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS        NAMES
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY      TAG          IMAGE ID        CREATED        SIZE
multi-stage     v1          424b6f2b8930   About an hour ago  273MB
single-stage    v1          84265953cf40   2 hours ago    465MB
waseem-apache   v1          d1c494f0eac8   2 hours ago    117MB
alpine-tomcat   latest      1939b2f8bfda   2 hours ago    296MB
nginx-custom    v1          83b9f4c22af3   3 hours ago    161MB
nginx           latest      5cdef4ac3335   37 hours ago    161MB
[root@ip-172-31-20-146 ~]# docker tag multi-stage:v1 waseem08/amazon-nginx:v1
[root@ip-172-31-20-146 ~]# docker images
REPOSITORY      TAG          IMAGE ID        CREATED        SIZE
multi-stage     v1          424b6f2b8930   2 hours ago    273MB
waseem08/amazon-nginx  v1          424b6f2b8930   2 hours ago    273MB
single-stage    v1          84265953cf40   2 hours ago    465MB
waseem-apache   v1          d1c494f0eac8   2 hours ago    117MB
alpine-tomcat   latest      1939b2f8bfda   2 hours ago    296MB
nginx-custom    v1          83b9f4c22af3   3 hours ago    161MB
nginx           latest      5cdef4ac3335   37 hours ago    161MB
[root@ip-172-31-20-146 ~]# docker push waseem08/amazon-nginx
```

Step:3

—> push that tag image to your docker gui

```
docker push waseem08/amazon-nginx:v1
```

```
[root@ip-172-31-20-146 ~]# docker push waseem08/amazon-nginx:v1
The push refers to repository [docker.io/waseem08/amazon-nginx]
ce35957d229a: Pushed
1afb855eb049: Pushed
1eb3ad63f844: Mounted from library/eclipse-temurin
a139bdf95402: Mounted from library/eclipse-temurin
69a2ceab248d: Mounted from library/eclipse-temurin
1209d2923796: Mounted from library/eclipse-temurin
123a078714d5: Mounted from library/eclipse-temurin
v1: digest: sha256:927c61434bf7eecd4ae64ac71e00bbfb1dc0c5a13cd5849acea8264751ec52cb size: 1786
[root@ip-172-31-20-146 ~]#
```

Step:4

—> now check your docker gui —> repository—> images

The screenshot shows the Docker Hub interface for the repository **waseem08/amazon-nginx**. The **Image Management** tab is selected, displaying a table of image tags. The table has columns for Digest, Tags, Media type, OS/ARCH, Size, Last pushed, Last pulled, and Status. A single row is visible for tag **v1** with digest **sha256:927c61434bf7**, media type **Image**, OS/ARCH **linux/amd64**, size **109.4 MB**, last pushed **1 minute** ago, last pulled **less than 1 day** ago, and status **Active**.

Digest	Tags	Media type	OS/ARCH	Size	Last pushed	Last pulled	Status
sha256:927c61434bf7	v1	Image	linux/amd64	109.4 MB	1 minute	less than 1 day	Active

The screenshot shows the Docker Hub interface for the repository **waseem08/amazon-nginx**, specifically the **Tags** tab. It displays a table of image tags with columns for TAG, Digest, OS/ARCH, Vulnerabilities, Last pull, and Compressed size. A single row is visible for tag **v1** with digest **927c61434bf7**, OS/ARCH **linux/amd64**, vulnerabilities **4 43 46 17 0**, last pull **less than 1 day** ago, and compressed size **109.39 MB**.

TAG	Digest	OS/ARCH	Vulnerabilities	Last pull	Compressed size
v1	927c61434bf7	linux/amd64	4 43 46 17 0	less than 1 day	109.39 MB

7.Implement solution to scan images when pushed to AWS ECR.

STEP 1:

Create an ECR Repository

```
aws ecr create-repository \  
  --repository-name my-ecr-repo \  
  --region eu-north-1
```

```
[root@ip-172-31-20-146 ~]# aws ecr describe-repositories \  
  --repository-names my-ecr-repo \  
  --region eu-north-1  
{  
  "repositories": [  
    {  
      "repositoryArn": "arn:aws:ecr:eu-north-1:207662791773:repository/my-ecr-repo",  
      "registryId": "207662791773",  
      "repositoryName": "my-ecr-repo",  
      "repositoryUri": "207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo",  
      "createdAt": "2026-02-06T13:01:49.355000+00:00",  
      "imageTagMutability": "MUTABLE",  
      "imageScanningConfiguration": {  
        "scanOnPush": true  
      },  
      "encryptionConfiguration": {  
        "encryptionType": "AES256"  
      }  
    }  
  ]  
}
```

Step 2: Login to AWS ECR (if not already)

```
aws ecr get-login-password --region eu-north-1 \  
| docker login --username AWS --password-stdin  
207662791773.dkr.ecr.eu-north-1.amazonaws.com  
Expected:
```

Login Succeeded

```
[root@ip-172-31-20-146 ~]# aws ecr get-login-password --region eu-north-1 \
| docker login --username AWS --password-stdin 207662791773.dkr.ecr.eu-north-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-20-146 ~]#
```

Step 3: Tag an existing image for ECR

Use one of your existing images (example: `multi-stage:v1`):

```
docker tag multi-stage:v1 \
207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo:v1
Verify:
```

```
docker images
```

```
[root@ip-172-31-20-146 ~]# docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo	v1	424b6f2b8930	2 hours ago	273MB
multi-stage	v1	424b6f2b8930	2 hours ago	273MB

Step 4: Push image to AWS ECR

```
docker push 207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo:v1
```

✓ Image pushed

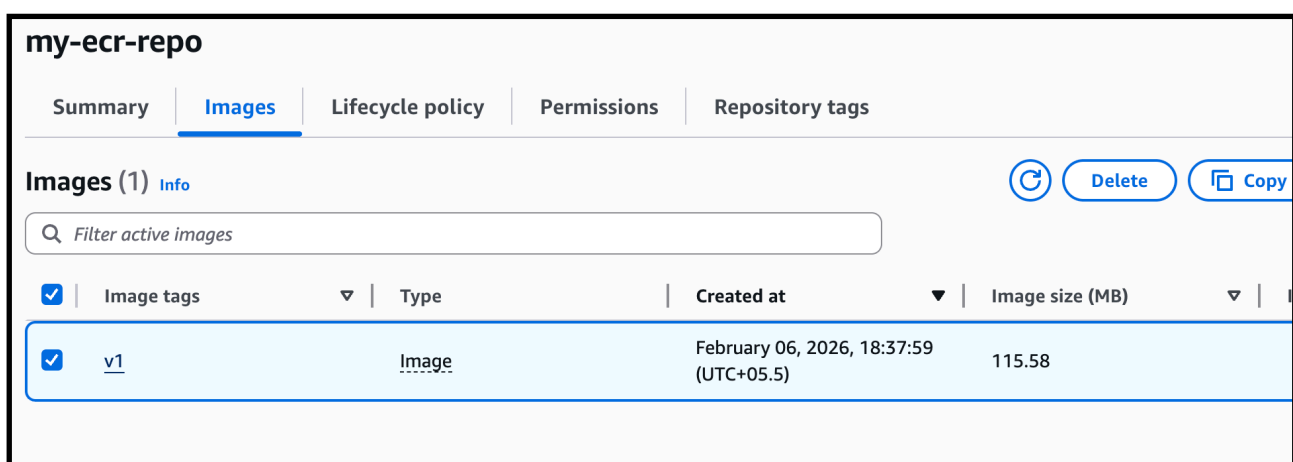
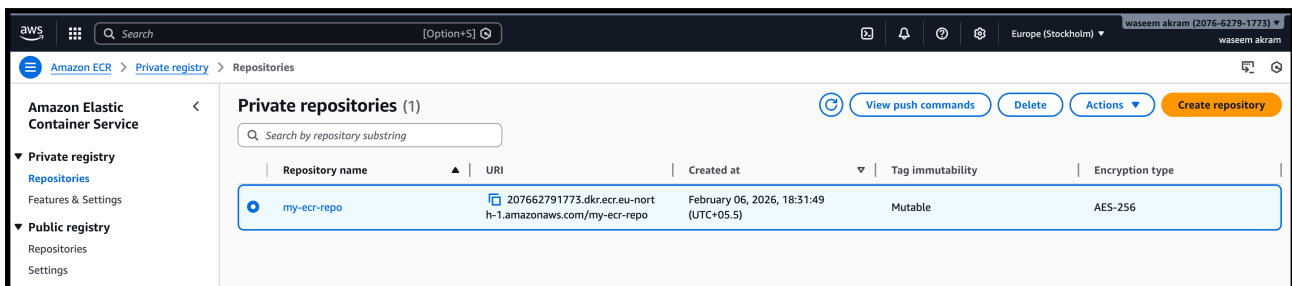
✓ Scan automatically triggered (because `scanOnPush=true`)

```
[root@ip-172-31-20-146 ~]# docker push 207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo:v1
The push refers to repository [207662791773.dkr.ecr.eu-north-1.amazonaws.com/my-ecr-repo]
ce35957d229a: Pushed
1afb855eb049: Pushed
1eb3ad63f844: Pushed
a139bdf95402: Pushed
69a2ceab248d: Pushed
1209d2923796: Pushed
123a078714d5: Pushed
v1: digest: sha256:9ab73aa71f80a8f2221a2fabd9f0fee44ab5069c406bd29963e61b72f40ab147 size: 1786
[root@ip-172-31-20-146 ~]#
```

Check in AWS Console

Path:

AWS Console → ECR → my-ecr-repo → Images → Scan findings



sha256:9ab75aa7180ab1221a21ab0910ree44ab5069c406bd29963ee1b72140ab147

General information

Artifact type

Image

Last recorded pull time

-

Repository

my-ecr-repo

Size (MB)

115.58

Pushed at

February 06, 2026, 18:37:59 (UTC+05.5)

Image status

Active

Scanning and vulnerabilities

Scan

Status

Complete

The scan was completed successfully.

Scan completed at

February 06, 2026, 18:38:10 (UTC+05.5)

Vulnerability source updated at

February 06, 2026, 18:38:10 (UTC+05.5)

Critical

0

High

0

Medium

0

Low

0

Info

0

8.Create a Jenkins pipeline to create a docker image and push the image to Docker hub.

Login to your Jenkins server gui

—>make sure you have docker—version

Step2:

Install plugins

Jenkins / Manage Jenkins / Plugins

Plugins

Updates 33

Available plugins

Installed plugins

Advanced settings

Q docker

Install

Install

Install	Name ↓	Released	Health
☒	Docker 1308.vff6e33248305 Cloud Providers Cluster Management docker This plugin integrates Jenkins with Docker	2 mo 17 days ago	100
☒	Docker Commons 457.v0f62a_94f11a_3 Library plugins (for use by other plugins) docker Provides the common shared functionality for various Docker-related plugins.	8 mo 12 days ago	93
☒	Docker Pipeline 634.vedc7242b_edda_7 pipeline DevOps Deployment docker Build and use Docker containers from pipelines. This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.	3 mo 25 days ago	76

—> make sure the docker is available in that server

Step3: create credentials of docker hub username password



Step3:

Create a repo

Add

docker-pipeline/

├── Jenkinsfile

├── Dockerfile

└── index.html

Jenkinsfile

```
Waseema761 / docker-pipeline
<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings
main docker-pipeline / Jenkinsfile
Waseema761 Update Jenkinsfile
Code Blame 51 lines (44 loc) · 1.15 KB
1 pipeline {
2   agent any
3
4   environment {
5     IMAGE_NAME = "waseem08/myapp"
6     IMAGE_TAG = "v1"
7     DOCKERHUB_CREDS = "dockerhub-creds"
8   }
9
10  stages {
11
12    stage("Checkout Code") {
13      steps {
14        checkout scm
15      }
16    }
17
18    stage("Build Docker Image") {
19      steps {
20        sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
21      }
22    }
23
24    stage("Docker Hub Login") {
25      steps {
26        withCredentials([usernamePassword(
27          credentialsId: DOCKERHUB_CREDS,
28          usernameVariable: 'DOCKER_USER',
29          passwordVariable: 'DOCKER_PASS'
30        )]) {
31          sh "echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin"
32        }
33      }
34    }
35  }
36 }
```

```
pipeline {

    agent any


    environment {

        IMAGE_NAME = "waseem08/myapp"

        IMAGE_TAG = "v1"

        DOCKERHUB_CREDS = "dockerhub-creds"

    }


    stages {

        stage("Checkout Code") {

            steps {

                checkout scm

            }

        }


        stage("Build Docker Image") {

            steps {

                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."

            }

        }


        stage("Docker Hub Login") {

            steps {
```

```
withCredentials([usernamePassword(  
  
    credentialsId: DOCKERHUB_CREDS,  
  
    usernameVariable: 'DOCKER_USER',  
  
    passwordVariable: 'DOCKER_PASS'  
  
)]) {  
  
    sh "echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin"  
  
    }  
  
    }  
  
}
```

```
stage("Push Image to Docker Hub") {  
  
    steps {  
  
        sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"  
  
    }  
  
    }  
  
}
```

```
post {  
  
    success {  
  
        echo "✅ Image successfully pushed to Docker Hub"  
  
    }  
  
    failure {  
  
        echo "❌ Pipeline failed"  
  
    }  
  
    }  
  
}
```

Dockerfile

Waseema761 / docker-pipeline

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security

main

docker-pipeline / Dockerfile

Waseema761

Add Dockerfile to serve index.html with Nginx

Code

Blame

2 lines (2 loc) · 67 Bytes

1

FROM

nginx:latest

2

COPY

index.html /usr/share/nginx/html/index.html

index.htm

main

docker-pipeline / index.html

Waseema761

Create index.html

Code

Blame

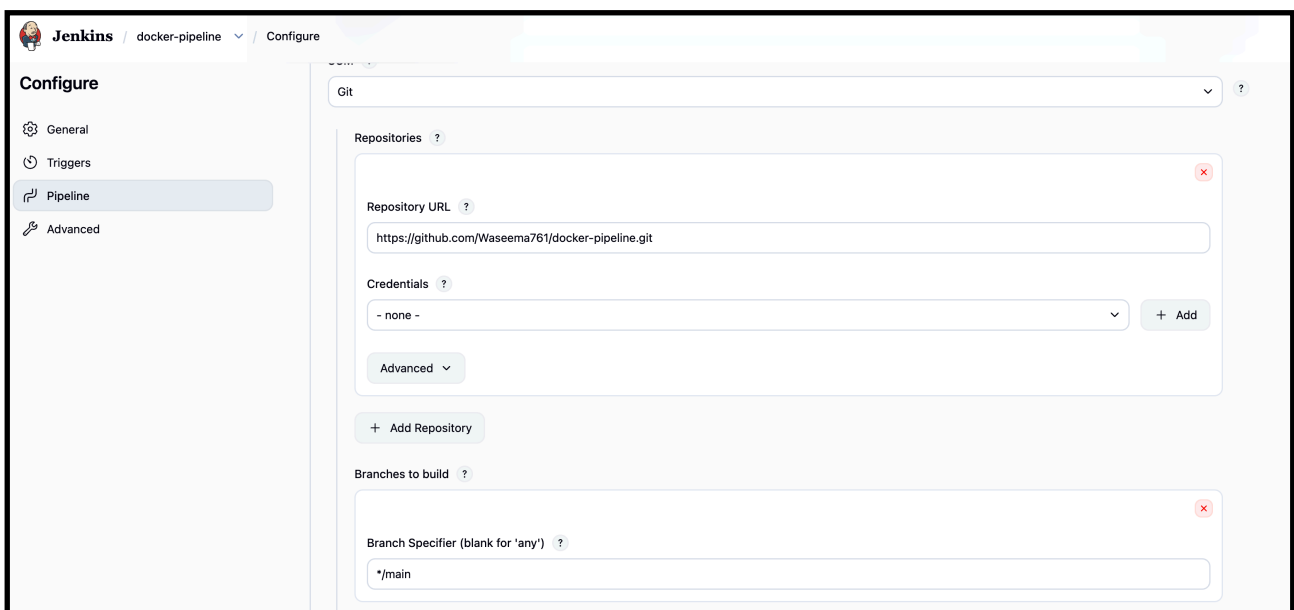
1 lines (1 loc) · 54 Bytes

1

<h1>Built & pushed by Jenkins using SCM pipeline</h1>

Create Pipeline

New Item → Pipeline → Name: **docker-pipeline**



The screenshot shows the Jenkins web interface for configuring a new pipeline named 'docker-pipeline'. The left sidebar contains the 'Configure' menu with options for General, Triggers, Pipeline, and Advanced. The 'Pipeline' option is selected. The main configuration area is titled 'Configure' and shows the 'Git' provider selected. Under 'Repositories', the 'Repository URL' is set to 'https://github.com/Waseema761/docker-pipeline.git'. The 'Credentials' dropdown is set to '- none -'. There is an 'Advanced' dropdown and an '+ Add Repository' button. Under 'Branches to build', the 'Branch Specifier (blank for \'any\')' is set to '*/main'.

—-> attach the repo url

Save

—> now build run

Jenkins / docker-pipeline

Status ✓ docker-pipeline

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Builds >

Filter

Today

✓ #5 2:26 PM

Stage View

Average stage times:
(full run time: ~9s)

	Declarative: Checkout SCM	Checkout Code	Build Docker Image	Docker Hub Login	Push Image to Docker Hub	Declarative: Post Actions
#5 Feb 06 19:56 No Changes	166ms	255ms	3s	390ms	4s	55ms
#4 Feb 06 19:54 No Changes						

Permalinks

- Last build (#5), 6 min 29 sec ago

—-> check your docker hub

waseem08
Docker Personal

Repositories

Hardened Images

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / myapp / General

waseem08/myapp

Last pushed 1 minute ago · ☆0 · ↓0

Add a description

Add a category

General Tags Image Management Collaborators Webhooks Settings

Tags

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
v1		Image	less than 1 day	2 minutes

See all

DOCKER SCOUT INACTIVE
[Activate](#)

Build with
Docker Build Cloud

Accelerate image build with Docker Build Cloud.

Docker Build Cloud e infrastructure with d

Get faster builds thro platform support, an infrastructure.

—> we can see we created waseem08/myapp
With v1 image

